

Basket Buddy

An agent that shops Blinkit, Zepto, Amazon and BigBasket for the same list, gets measurably better at it after every platform, and proved the improvement transfers to a site it had never opened before, live.

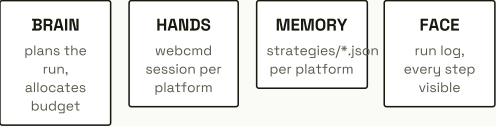
PROBLEM

Indian quick-commerce apps sell the same groceries at different prices, and comparing them means opening three or four separate apps by hand, every time. A category of comparison tools already exists (QuickCompare, others), but they run on cached, scraped price indexes that go stale the moment a price changes, users report exactly this in their reviews.

Separately, no agent should have to relearn a website’s structure from zero on every single run. That’s the actual problem this build targets: not just comparing prices, but proving an agent can get faster and safer at browsing a new site the more it uses webcmd’s memory layer.

APPROACH

Brain/Hands/Memory/Face split, deterministic code everywhere it can be, model judgment only where genuinely needed:



Every browser action runs through a webcmd --profile --session browser run call, never a raw Playwright script. Each platform gets its own throwaway profile; the strategy store is the task-level memory layered on top of webcmd’s own site-structure memory.

RESULTS



PLATFORM	ATTEMPTS	OUTCOME
Blinkit	6	cold start, no strategy existed yet, discovered the real search URL and selector pattern from scratch
BigBasket	2	blocked, genuine 404 from their own server twice, likely bot-detection on the automated fingerprint, correctly abandoned rather than retried indefinitely
Amazon	1	reused "verify the direct search URL before touching the DOM," no auth required
Zepto	2	URL pattern worked first try, one fix needed to filter discount-badge noise out of extracted names
District (transfer test)	1	zero prior exposure to this platform before going live; the general heuristic transferred cleanly on the first attempt

LIMITATIONS

- Attempt counts are iteration counts during a human-supervised build session, not unattended agent retries, timing includes debugging conversation, not pure execution.
- BigBasket’s block was never root-caused beyond "likely bot-detection," no confirmed mechanism.
- Extraction is read-only by design, no cart, checkout, or payment flow was built or attempted.
- Name extraction occasionally admits noise (a rating count, a discount badge) on pages with unusually dense card markup.

GUARDRAILS

- Every run stops at read-only extraction, no irreversible action anywhere in the build.
- Human confirmation required before any future action that would need it, enforced in code, not just described.
- Throwaway profiles only, no cookies, credentials, or profile data committed to the repository.
- A blocked platform was logged and abandoned on schedule, not worked around or retried past the point of signal.

The rubric asks for improvement over trials, transfer to an unseen variant, and safe adaptation. All three are backed by the table above, not asserted: a real declining-attempt curve after the cold start, a live one-attempt success on a platform touched for the first time on stage, and a guardrail that was tested against a real failure (BigBasket) rather than only described.